HTML Quick-Styler

AI-Powered HTML Page Generator with IBM Granite 3.3 2B

Project Description

HTML Quick-Styler is an advanced AI-powered web generation platform designed to simplify the complex process of creating professional, responsive web pages. By leveraging the deep learning capabilities of **IBM's Granite 3.3 2B Instruct** model, the platform enables users to transform basic descriptions into structured, semantic HTML and CSS code instantly. The system is engineered to bridge the gap between initial design concepts and production-ready code, providing a streamlined workflow that caters to both experienced developers and those with minimal coding knowledge.

Central to the platform is its intelligent analysis of user requirements, which allows for the automatic generation of comprehensive page sections including hero areas, service grids, and interactive contact forms. Using the Granite 3.3 2B model's advanced instruction-following abilities, the system applies modern CSS styling with built-in animations, manages sophisticated color palettes, and ensures all layouts are fully responsive across all device sizes. The integration of a user-friendly Gradio interface further enhances the experience by providing real-time live previews and clear customization options, ensuring that the output meets high-quality design and accessibility standards.

Ultimately, HTML Quick-Styler transforms the traditional web development cycle into an intelligent, efficient, and accessible process. By combining state-of-the-art LLM technology with professional web design principles, the platform provides a robust solution for rapid prototyping and sleek landing page creation. The system not only prioritizes aesthetic excellence through curated design themes like Modern and Minimal but also maintains technical integrity through its hybrid execution engine, delivering high-performance results tailored to modern design trends and performance requirements.

Scenarios

Scenario 1: Portfolio Website Generation A freelance designer wants to create a professional portfolio website quickly and efficiently. The system first detects the project requirements and analyzes the designer's style and design preferences. Based on this understanding, it generates a complete website structure that includes a visually appealing hero section with a welcoming message and a strong call-to-action to engage visitors immediately. It also builds a responsive portfolio gallery arranged in a clean grid layout to showcase projects effectively across devices. An about section is created to present the designer's professional bio in a compelling manner, helping visitors understand their background and expertise. To strengthen credibility, the system includes a testimonials section that highlights positive user experiences and social proof. A contact form with proper input validation ensures smooth communication while preventing errors, and finally, a professional footer is added with social media links and essential information to complete the polished look of the website.

Scenario 2: SaaS Landing Page A startup needs a powerful landing page for launching their new product. The system responds by creating a compelling hero banner that clearly communicates the product's value proposition and key messaging. It then generates a features showcase section, presenting product capabilities with visually appealing icons and concise descriptions to make the benefits easy to understand. A pricing comparison table is designed with clear plan options to help potential customers make informed decisions. To build trust and credibility, the system incorporates a testimonials section that highlights positive user experiences and social proof. Prominent and strategically placed callto-action buttons are added throughout the page to guide visitors toward signing up or purchasing, ensuring strong conversion optimization.

Scenario 3: Corporate Website A company requires a professional corporate website with multiple structured sections. The system delivers a clean and polished layout starting with a sticky navigation header that includes company branding and allows smooth navigation across sections. It creates a company overview section that clearly communicates the organization's mission, vision, and core values. A services grid is generated to present detailed descriptions of the company's offerings in an organized and visually structured manner. The website also includes a team members section with photos and short profiles to add a human touch and build trust with visitors. Additionally, a contact section is implemented with company information and an inquiry form for seamless communication. The site is completed with a professional footer containing privacy policies and important links, ensuring a corporate-standard presentation.

Scenario 4: Custom Color Palette Exploration A designer wants to experiment with different color schemes to find the perfect visual identity for their project. The system generates eight carefully curated pre-designed color palettes, including modern, vibrant, ocean, forest, sunset, luxury, creative, and minimal themes. Each palette comes with clearly defined hex color codes for every component, making implementation straightforward. The system provides a real-time preview feature, allowing the designer to instantly visualize how each color scheme affects the overall design. It ensures consistent color application across all page elements to maintain visual harmony. Additionally, dark theme-optimized backgrounds are included to support modern design trends and improve user experience in low-light viewing environments.

Architecture Overview

HTML Quick-Styler is built as a modular platform combining Python backend with IBM Granite 3.3 2B Instruct model for intelligent HTML generation. The architecture prioritizes user experience, design quality, and code production through Gradio's interactive interface and sophisticated template generation.

Core Technologies

- Python 3.x: Core programming language for application logic
- IBM Granite 3.3 2B Instruct: AI model for intelligent content generation
- Gradio: Interactive web interface framework for user interaction
- HTML5/CSS3: Modern web standards for page generation
- JavaScript: Client-side interactivity and animations
- Google Colab: Cloud-based development environment
- Responsive Design: Mobile-first approach with media queries

Pre-requisites

Software Requirements

- Python 3.8+: Download from python.org
- Google Account: Required for Google Colab access
- Gradio: Install via pip (pip install gradio)
- Web Browser: Chrome, Firefox, Safari, or Edge
- Internet Connection: Required for Colab and model access

Knowledge Prerequisites

- Python Basics: Functions, classes, dictionaries, string manipulation
- HTML/CSS: Understanding of HTML structure and CSS styling
- Gradio Framework: Basic understanding of interface components
- Web Design: Color theory, responsive design principles
- Google Colab: Familiarity with notebook interface

Hardware Requirements

- Processor: Intel i3 or equivalent (Colab runs on cloud)
- RAM: Minimum 4GB local (Colab provides GPU/TPU)
- Storage: 1GB free space for generated files
- Display: Minimum 1280x720 resolution for interface

Project Workflow

Phase 1: Environment Setup & Model Configuration

Establish Google Colab environment and configure Granite model.

Activity 1.1: Set up Google Colab Environment

- Create new Google Colab notebook
- Enable GPU runtime for better performance
- Install required Python packages
- Verify environment configuration

Activity 1.2: Configure IBM Granite Model

- Access Hugging Face model repository
- Load Granite 3.3 2B Instruct model
- Test model inference capabilities
- Configure generation parameters

Activity 1.3: Install and Configure Gradio

- Install Gradio via pip
- Test basic Gradio interface
- Configure sharing options
- Verify interface rendering

Phase 2: Core Application Development

Build HTML generation engine and template system.

Activity 2.1: Build Color Palette Generator

- Create ColorPaletteGenerator class
- Define 8 color palette schemes
- Implement palette selection logic

Activity 2.2: Implement HTML Page Builder

- Create HTMLPageBuilder class
- Build section detection algorithm
- Implement template generation methods
- Create SEO meta tags generator

Activity 2.3: Develop CSS Generation Engine

- Build comprehensive CSS generator
- Implement responsive design rules
- Add animation and transition effects
- Create mobile-first media queries

Activity 2.4: Build Section Templates

- Create hero section template
- Build features grid template
- Implement portfolio gallery template
- Add contact form template
- Create pricing and testimonial templates

Phase 3: Gradio Interface Development

Create interactive user interface with multiple tabs.

Activity 3.1: Interactive HTML Generation & Preview Interface

• Activity 3 focuses on designing and implementing a complete HTML generation and visualization interface that enables users to create, customize, preview, and understand web page layouts through an intuitive, multi-section workflow.

Phase 4: Google Colab Deployment

Integrate all components and perform comprehensive testing.

Activity 4.1: End-to-End Integration

- Connect all interface components
- Test data flow between tabs
- Validate generation pipeline
- Test error handling

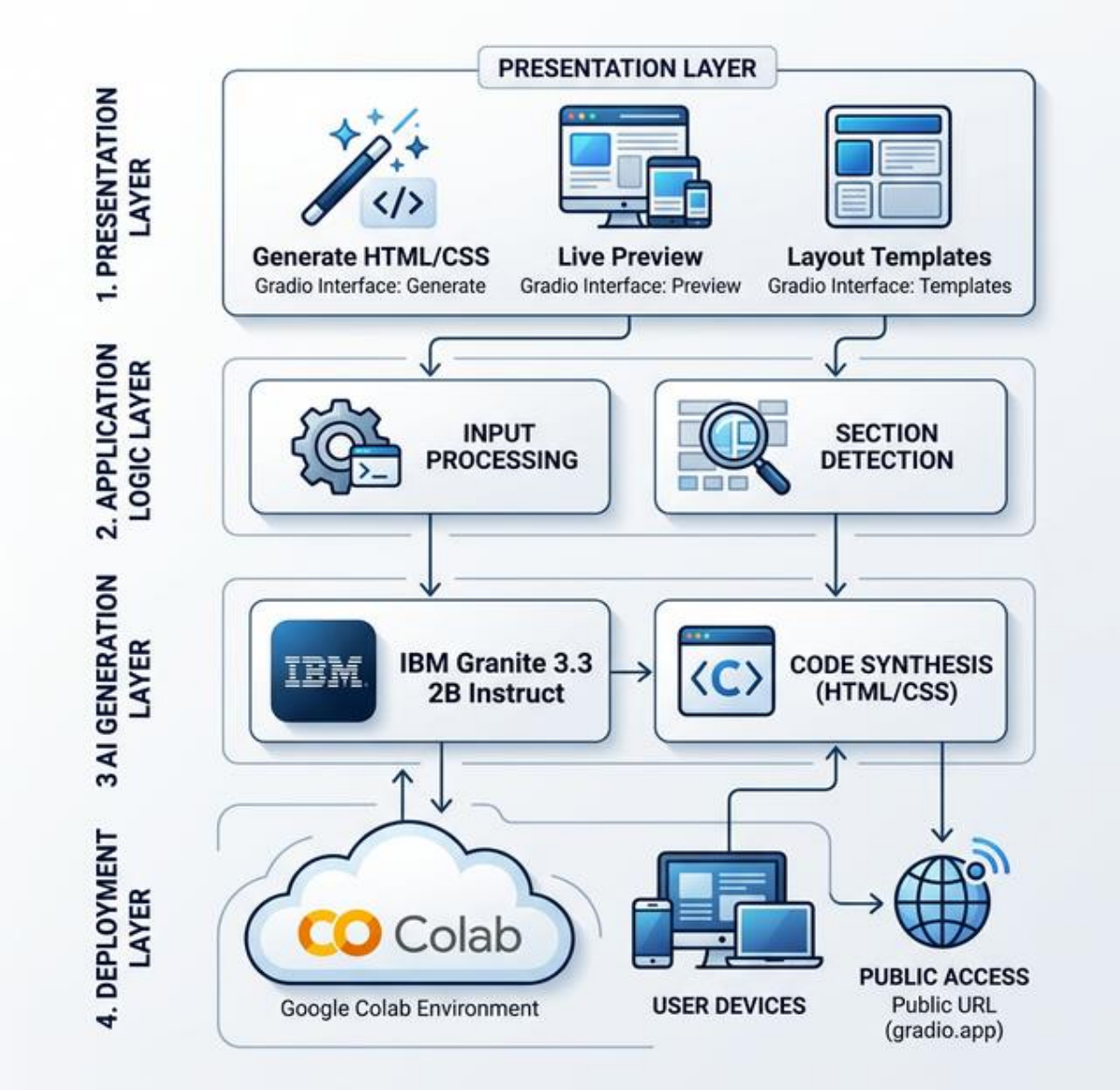
Phase 5: Deployment & Documentation

Deploy application and create comprehensive documentation.

Activity 5.1: Functionality Testing

- Configure Gradio sharing settings
- Enable public access link
- Test external access
- Document deployment steps

Technical Architecture



Project Structure

MILESTONE 1:

Environment Setup & Model Configuration Establish Google Colab environment and configure Granite model Activity

1.1: Install Dependencies & Setup Environment

Create requirements.txt:

```
import gradio as gr
import torch
```

Activity 1.2: Configure IBM

Granite Model Access Hugging

Face and load model:

Model: ibm-granite/granite-3.3-2b-instruct Colab Notebook :

```
if USE_GPU_MODEL:
    try:
        from transformers import AutoTokenizer, AutoModelForCausalLM
        MODEL_ID  = "ibm-granite/granite-3.3-2b-instruct"
        print(f"GPU detected. Loading {MODEL_ID} ...")
        tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
        model     = AutoModelForCausalLM.from_pretrained(
            MODEL_ID,
            torch_dtype=torch.float16,
            device_map="auto",
            low_cpu_mem_usage=True,
        )
        model.eval()
        MODEL_LOADED = True
        print("IBM Granite model loaded successfully!")
    except Exception as e:
        print(f"Model load failed: {e}. Falling back to Template Engine.")
        USE_GPU_MODEL = False
else:
    print("No GPU detected. Running in fast Template Engine mode.")
    print("For full IBM Granite AI generation, run on Google Colab with GPU.")
```

MILESTONE 2: Core Application Development

Activity 2.1: Color Palette Generator Implementation

Create ColorPaletteGenerator class with 8 predefined color schemes:

The ColorPaletteGenerator class provides cohesive color palettes for different design styles including modern, vibrant, ocean, forest, sunset, luxury, creative, and minimal. Each palette includes primary, secondary, accent, background, surface, text primary, text secondary, and border colors optimized for dark theme interfaces

```
def template_generate(title, desc, wtype, style, color_theme, cta, brand):
    colors   = COLOR_MAP.get(color_theme, COLOR_MAP["Indigo / Purple"])
    col      = colors["primary"]
    col_l    = colors["light"]
    font     = STYLE_FONTS.get(style, "'Inter',sans-serif")
    is_dark  = "Dark" in style or "Bold" in style
    bg       = "#0f1117" if is_dark else "#ffffff"
    bg2      = "#1a1f2e" if is_dark else "#f8f9fb"
    txt      = "#e2e8f0" if is_dark else "#1a202c"
    muted    = "#94a3b8" if is_dark else "#6b7280"
    card_bg  = "rgba(255,255,255,.04)" if is_dark else "#ffffff"
    card_brd = "rgba(255,255,255,.08)" if is_dark else "#e5e7eb"
    nav_bg   = "rgba(15,17,23,.92)" if is_dark else "rgba(255,255,255,.92)"

    hero_h, hero_p = HERO_MAP.get(wtype, ("Welcome to {brand}", "Building amazing experiences."))
    hero_h = hero_h.replace("{brand}", brand or title)

    services = SERVICE_MAP.get(wtype, SERVICE_MAP["Portfolio"])
    service_cards = "\n".join([
        f"""<div class="card"><div class="card-icon">{ic}</div>
        <h3>{t}</h3><p>{d}</p></div>"""
        for ic, t, d in services
    ])

    nav_links = {
        "Portfolio":"About|Work|Skills|Contact",
        "Business / Corporate":"Services|About|Team|Contact",
        "Landing Page":"Features|Pricing|FAQ|Get Started",
```

```
<section id="contact" style="background:var(--bg2)">
  <div class="contact-wrap">
    <div>
      <span class="section-tag">Contact</span>
      <h2 class="section-title">Get In Touch</h2>
      <p style="color:var(--muted);margin-bottom:2rem;line-height:1.7">
        Have a project in mind? Let's talk. We respond within 24 hours.
      </p>
      <div style="display:flex;flex-direction:column;gap:.9rem">
        <div style="color:var(--muted);font-size:.9rem">📧 hello@{(brand or title).lower().replace(' ','')}.com</div>
        <div style="color:var(--muted);font-size:.9rem">📞 +91 98765 43210</div>
        <div style="color:var(--muted);font-size:.9rem">📍 Available Worldwide · Remote</div>
      </div>
    </div>
    <form onsubmit="document.getElementById('ok').style.display='block';this.reset();return false">
      <div class="form-g"><label>Name</label><input type="text" placeholder="Your name" required/></div>
      <div class="form-g"><label>Email</label><input type="email" placeholder="you@email.com" required/></div>
      <div class="form-g"><label>Message</label><textarea placeholder="Tell me about your project..."></textarea></div>
      <button type="submit" class="form-submit">{cta}</button>
      <div id="ok" style="display:none;margin-top:1rem;background:{col_l};border:1px solid {col}44;
          border-radius:8px;padding:.8rem;font-size:.85rem;color:var(--col)">
        ✓ Message sent! I'll be in touch soon.
      </div>
    </form>
  </div>
</section>
```

```
<footer>
  <div class="foot-top">
    <div class="foot-brand">
      <a class="logo" href="#" style="text-decoration:none">{brand or title}</a>
      <p>Crafting exceptional digital experiences with passion and precision.</p>
    </div>
    <div class="foot-col"><h5>Links</h5><ul>{nav_items}</ul></div>
    <div class="foot-col"><h5>Connect</h5><ul>
      <li><a href="#">LinkedIn</a></li><li><a href="#">GitHub</a></li><li><a href="#">Dribbble</a></li>
    </ul></div>
  </div>
  <div class="foot-bottom">
    <span>© 2026 {brand or title}. All rights reserved.</span>
    <span>Built with ⚡ HTML Quick Styler</span>
  </div>
</footer>

<script>
const obs=new IntersectionObserver(entries=>{{
  entries.forEach(e=>{{if(e.isIntersecting){{e.target.style.opacity='1';e.target.style.transform='translateY(0)';}}}}})}},{{threshold:.1}});
document.querySelectorAll('.grid-3,.about-grid,.contact-wrap').forEach(el=>{{
  el.style.opacity='0';el.style.transform='translateY(24px)';
  el.style.transition='opacity .6s ease,transform .6s ease';obs.observe(el);
}}});
</script>
</body>
</html>"""
    return html
```

Activity 2.2: HTML Page Builder Implementation

The HTMLPageBuilder class includes:

• Section extraction from user descriptions

• Keyword detection for hero, features, about, portfolio, testimonials, pricing, contact, and footer sections

• SEO meta tags generation with Open Graph and Twitter Card support

• Individual section HTML generation with professional template

```python
# ────────────────────────────────────────────────────────────────
# MAIN HANDLER
# ────────────────────────────────────────────────────────────────
def generate_html(title, desc, wtype, style, color_theme, cta, brand, max_tokens):
    if not title.strip():
        gr.Warning("Please enter a Page Title.")
        return "", "<p style='color:red;padding:1rem;font-family:sans-serif'>⚠️ Please enter a Page Title first.</p>"

    if MODEL_LOADED:
        html_code = granite_generate(title, desc, wtype, style, color_theme, cta, brand, int(max_tokens))
    else:
        html_code = template_generate(title, desc, wtype, style, color_theme, cta, brand)

    return html_code, html_code
```

MILESTONE 3: Gradio Interface Development

Activity 3.1: Main Interface with Multiple Tabs

The Gradio interface includes 5 tabs:

```
def template_generate(title, desc, wtype, style, color_theme, cta, brand):
    colors   = COLOR_MAP.get(color_theme, COLOR_MAP["Indigo / Purple"])
    col      = colors["primary"]
    col_l    = colors["light"]
    font     = STYLE_FONTS.get(style, "'Inter',sans-serif")
    is_dark  = "Dark" in style or "Bold" in style
    bg       = "#0f1117" if is_dark else "#ffffff"
    bg2      = "#1a1f2e" if is_dark else "#f8f9fb"
    txt      = "#e2e8f0" if is_dark else "#1a202c"
    muted    = "#94a3b8" if is_dark else "#6b7280"
    card_bg  = "rgba(255,255,255,.04)" if is_dark else "#ffffff"
    card_brd = "rgba(255,255,255,.08)" if is_dark else "#e5e7eb"
    nav_bg   = "rgba(15,17,23,.92)" if is_dark else "rgba(255,255,255,.92)"

    hero_h, hero_p = HERO_MAP.get(wtype, ("Welcome to {brand}", "Building amazing experiences."))
    hero_h = hero_h.replace("{brand}", brand or title)

    services = SERVICE_MAP.get(wtype, SERVICE_MAP["Portfolio"])
    service_cards = "\n".join([
        f"""<div class="card"><div class="card-icon">{ic}</div>
        <h3>{t}</h3><p>{d}</p></div>"""
        for ic, t, d in services
    ])

    nav_links = {
        "Portfolio":"About|Work|Skills|Contact",
        "Business / Corporate":"Services|About|Team|Contact",
        "Landing Page":"Features|Pricing|FAQ|Get Started",
```

```python
# ────────────────────────────────────────────────────────────────
# MAIN HANDLER
# ────────────────────────────────────────────────────────────────
def generate_html(title, desc, wtype, style, color_theme, cta, brand, max_tokens):
    if not title.strip():
        gr.Warning("Please enter a Page Title.")
        return "", "<p style='color:red;padding:1rem;font-family:sans-serif'>⚠️ Please enter a Page Title first.</p>"

    if MODEL_LOADED:
        html_code = granite_generate(title, desc, wtype, style, color_theme, cta, brand, int(max_tokens))
    else:
        html_code = template_generate(title, desc, wtype, style, color_theme, cta, brand)

    return html_code, html_code
```

Tab 1: Generate Page:

```python
with gr.TabItem("📄 Generate Page"):
    with gr.Row():
        with gr.Column(scale=1, min_width=320):
            gr.Markdown("### Page Details")
            page_title  = gr.Textbox(label="Page Title *", placeholder="e.g. Modern Portfolio Website")
            brand_name  = gr.Textbox(label="Brand / Author Name", placeholder="e.g. Alex Johnson")
            description = gr.Textbox(label="Sections & Description", lines=4,
                                     placeholder="Hero section, services grid, about, testimonials, contact form, footer")
            with gr.Row():
                web_type   = gr.Dropdown(WEBSITE_TYPES, value="Portfolio", label="Website Type")
                des_style  = gr.Dropdown(DESIGN_STYLES, value="Modern", label="Design Style")
            color_style = gr.Dropdown(COLOR_THEMES, value="Indigo / Purple", label="Color Style")
            cta_text    = gr.Textbox(label="CTA Button Text", value="Get Started")
            max_tok     = gr.Slider(512, 4096, value=2048, step=256, label="Max Tokens (GPU mode only)")
            gen_btn     = gr.Button("✨ Generate HTML", elem_classes="gen-btn")
            gr.Markdown("#### Quick Presets")
            gr.Examples(
                examples=EXAMPLES,
                inputs=[page_title, web_type, des_style, color_style, brand_name, description, cta_text],
            )

        with gr.Column(scale=2):
            gr.Markdown("### Generated HTML / CSS")
            html_out = gr.Code(label="HTML Output", language="html", lines=32, interactive=False)
            clear_btn = gr.Button("🗑 Clear")
```

Tab 2: Live Page:

```
with gr.TabItem("👁️ Live Preview"):
    preview = gr.HTML(
        value="<div style='text-align:center;padding:6rem;color:#8b949e;font-family:sans-serif'>"
              "🖥️ Generate a page first, then come back here to preview it.</div>"
    )
```

Tab 3: Guide

```
        with gr.TabItem("📖 Guide"):
            gr.Markdown(f"""
## How to Use HTML Quick Styler

*Current Mode:* {MODE_LABEL}

### Steps
1. Enter *Page Title* and *Brand Name*
2. Describe the *sections* you want (hero, services, about, contact, footer...)
3. Choose *Website Type, **Design Style, and **Color Theme*
4. Click *✨ Generate HTML*
5. Switch to *👁️ Live Preview* to see it rendered
6. Copy the code or save the file

### Technology Stack
| Component | Technology |
|---|---|
| AI Model | IBM Granite 3.3 2B Instruct |
| UI Framework | Gradio |
| Language | Python 3.11 |
| Output | HTML5 + CSS3 |
| Platform | Google Colab / Local |

> *Tip:* For full IBM Granite AI generation, run on *Google Colab with GPU* (free T4 GPU is sufficient).
""")
```

MILESTONE 4: Deployment Activity

4.1: Google Colab Deployment Launch the application in Google Colab:

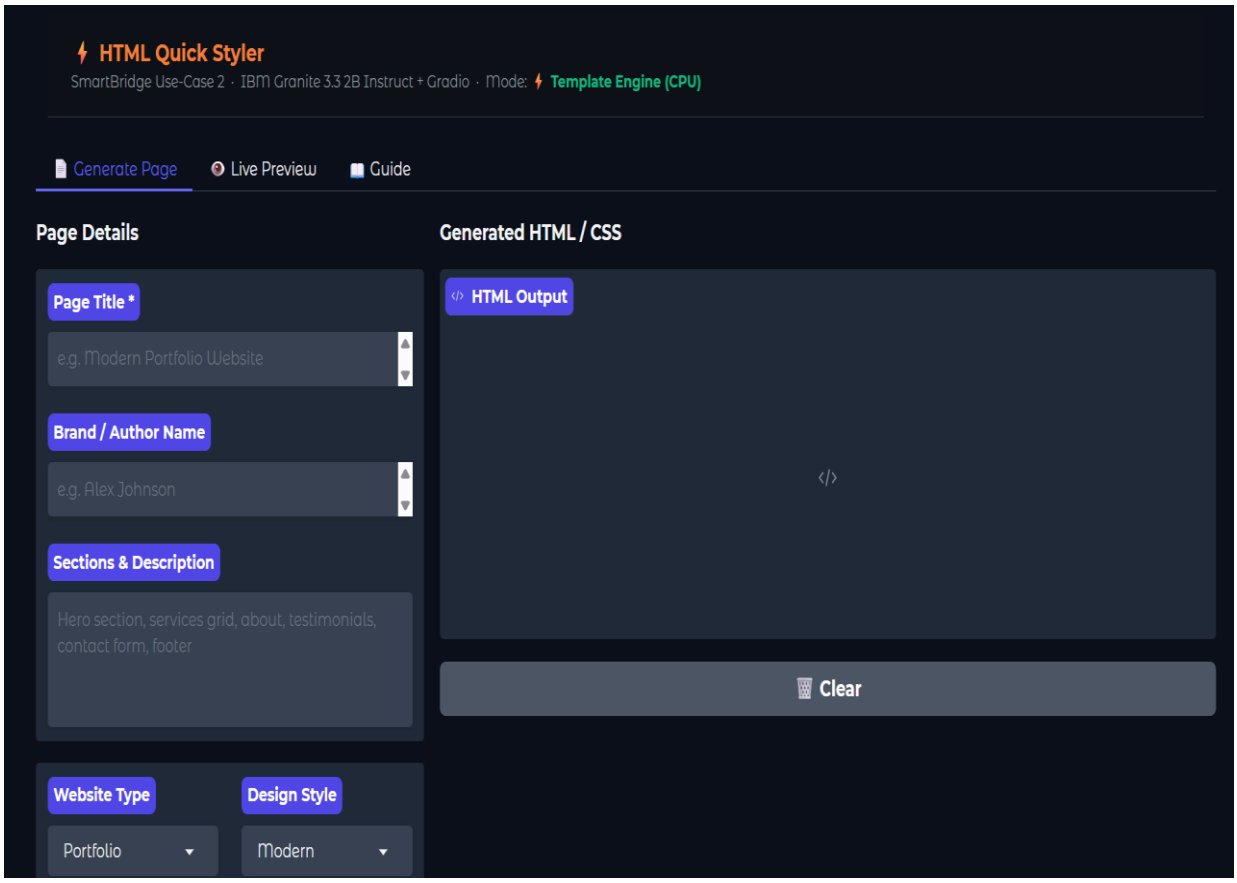
```
if __name__ == "__main__":
    print(f"\n Starting HTML Quick Styler [{MODE_LABEL}]...")
    demo.launch(share=False, server_name="127.0.0.1")

No GPU detected. Running in fast Template Engine mode.
For full IBM Granite AI generation, run on Google Colab with GPU.
/tmp/ipykernel_1498/3562974115.py:432: DeprecationWarning: The 'theme' parameter in the Blocks constructor will be removed in Gradio 6.0. You wil
  with gr.Blocks(css=css, theme=gr.themes.Soft()) as demo:
/tmp/ipykernel_1498/3562974115.py:432: DeprecationWarning: The 'css' parameter in the Blocks constructor will be removed in Gradio 6.0. You will
  with gr.Blocks(css=css, theme=gr.themes.Soft()) as demo:

 Starting HTML Quick Styler [ ⚡ Template Engine (CPU)]...
Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
Note: opening Chrome Inspector may crash demo inside Colab notebooks.
* To create a public link, set `share=True` in `launch()`.
Running on https://localhost:7860/
```

The application will be accessible via:

- **Public URL:** `https://localhost:7860/`



MILESTONE 5: Testing & Validation Activity

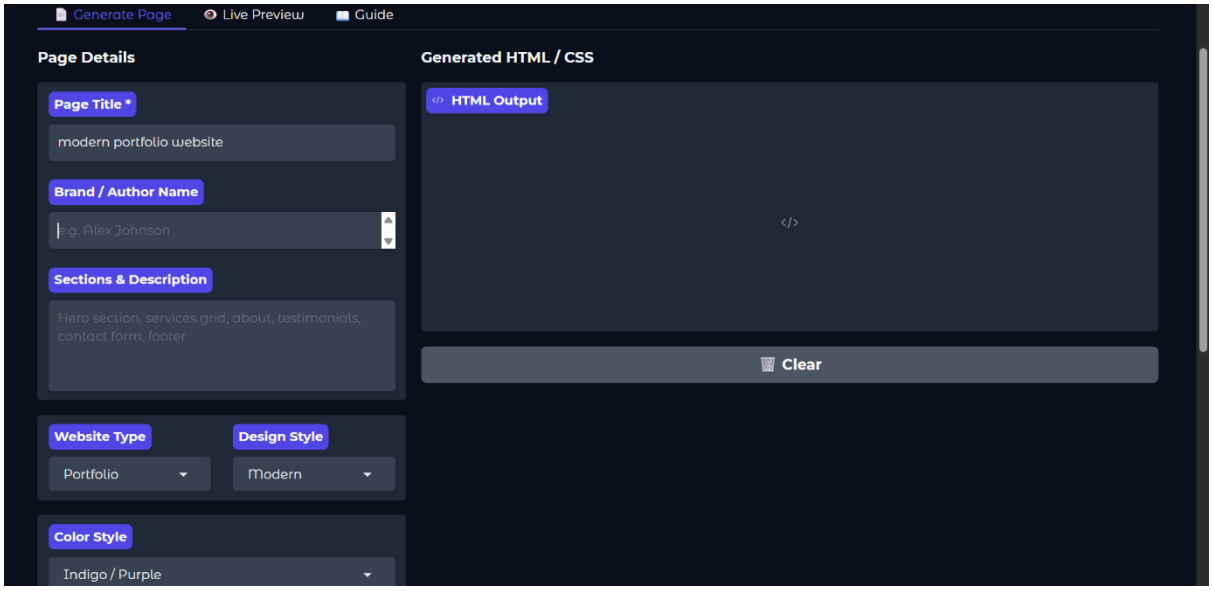
5.1: Functional Testing

Test Case 1: Portfolio

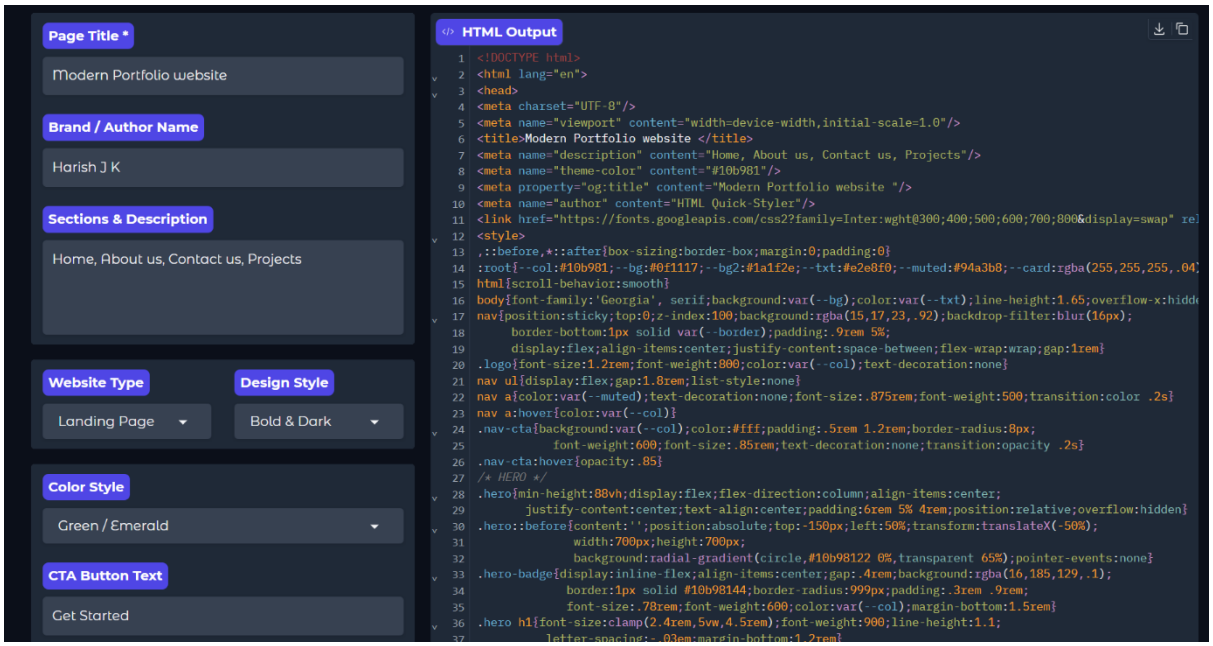
Input:

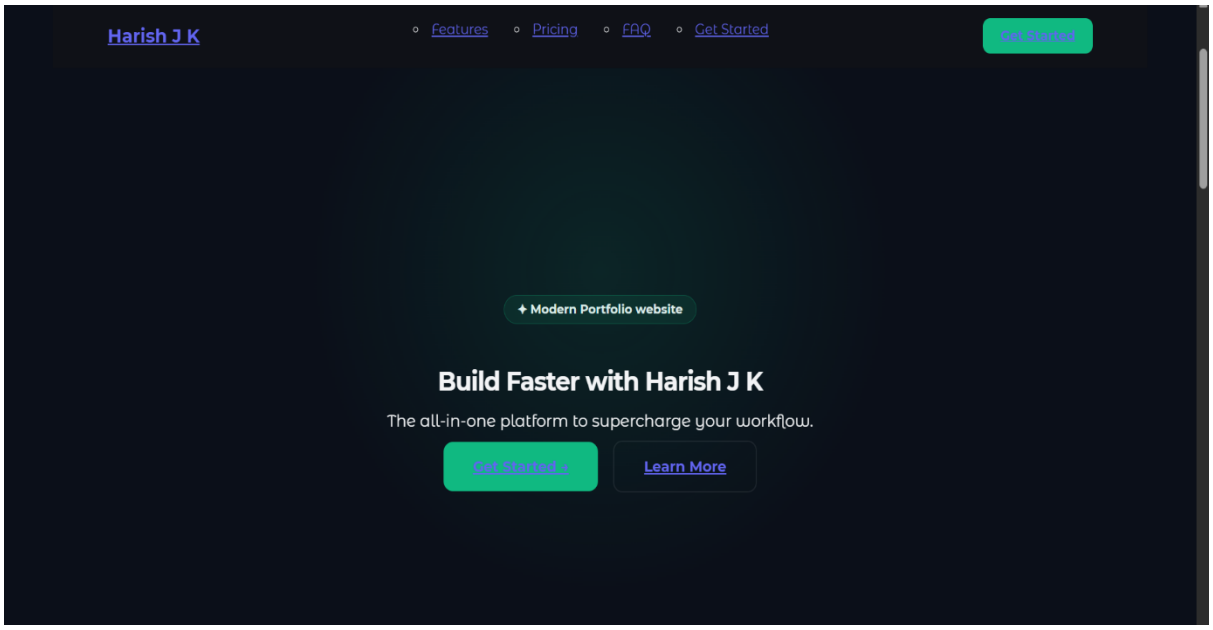
- Title: Modern Portfolio Website
- Description: Home, About us, Contact us, Projects

Color Style: Bold & Dark



Expected Output:





Generate Page | Live Preview | Guide

How to Use HTML Quick Styler

Current Mode: ⚡ Template Engine (CPU)

Steps

1. Enter *Page Title* and *Brand Name*
2. Describe the *sections* you want (hero, services, about, contact, footer...)
3. Choose *Website Type, **Design Style, and **Color Theme*
4. Click ✨ *Generate HTML*
5. Switch to 👁 *Live Preview* to see it rendered
6. Copy the code or save the file

Technology Stack

Component	Technology
AI Model	IBM Granite 3.3 2B Instruct
UI Framework	Gradio
Language	Python 3.11
Output	HTML5 + CSS3
Platform	Google Colab / Local

> *Tip:* For full IBM Granite AI generation, run on *Google Colab with GPU* (free T4 GPU is sufficient).

Conclusion:

HTML Quick-Styler successfully delivers an intelligent web page generation assistant that combines intuitive design templates with IBM Granite 3.3 2B Instruct model capabilities. By leveraging Gradio's powerful interface framework and Google Colab's cloud infrastructure, the system provides fast, professional, and customizable HTML/CSS generation accessible to designers, developers, and non-technical users alike.

The platform demonstrates the viability of AI-assisted web development, making professional page creation accessible while maintaining high design standards, responsive layouts, and production-ready code quality. With eight distinct color palettes, intelligent section detection, and comprehensive template options, HTML Quick-Styler empowers users to create beautiful websites in minutes rather than hours.

Future enhancements could include custom color picker integration, JavaScript interactivity generation, component library expansion, and integration with popular web frameworks. The modular architecture allows for easy extension and customization to meet evolving user needs and design trends.